

ESTRUTURA DE DADOS 2

Árvores Rubro-Negra

Red-Black Trees

Agenda do Seminário

Aula 1

Teoria: Fundamentos



Daniel · Lucas

Aula 2

Teoria: Operações e Complexidade



Chris · Rildo

Aula 3

Prática: Implementação



Rildo · Adilson

Aula 4

Prática: Hands-on e Quiz



Perla

O que é uma Árvore Rubro-Negra?

Definição: É uma árvore binária de busca onde cada nó tem uma cor (**vermelha ou preta**) e um conjunto de regras que garantem balanceamento aproximado.

Origem: Proposta por Guibas & Sedgwick (1978) como um framework dicromático para árvores balanceadas. O nome vem das canetas coloridas disponíveis no laboratório na época.

Relação com B-trees: uma Red-Black pode ser vista como uma B-tree 2-3-4 representada em dois níveis. Cada nó preto com filhos vermelhos equivalentes a um nó com múltiplas chaves.

As 5 Propriedades Rubro-Negra

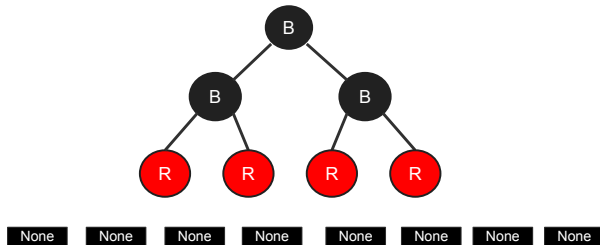
1 **Cor Binária** - Cada nó é vermelho ou preto

2 **Raiz Preta** - A raiz é sempre preta

3 **NILs Pretos** - Todos os nós NIL (folhas) são pretos

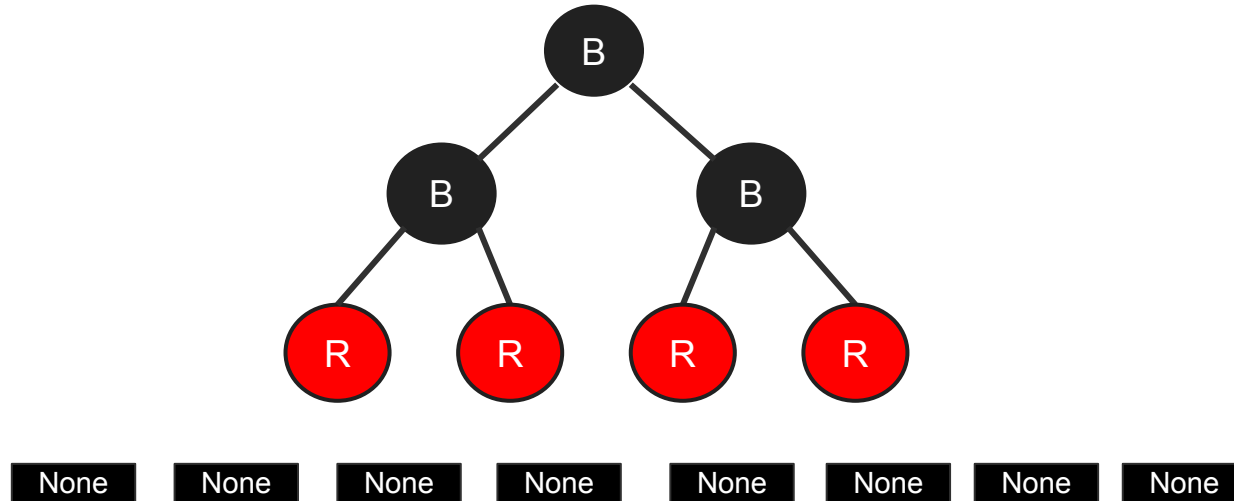
4 **Sem Vermelhos Adjacentes** - Nó vermelho não pode ter filho vermelho

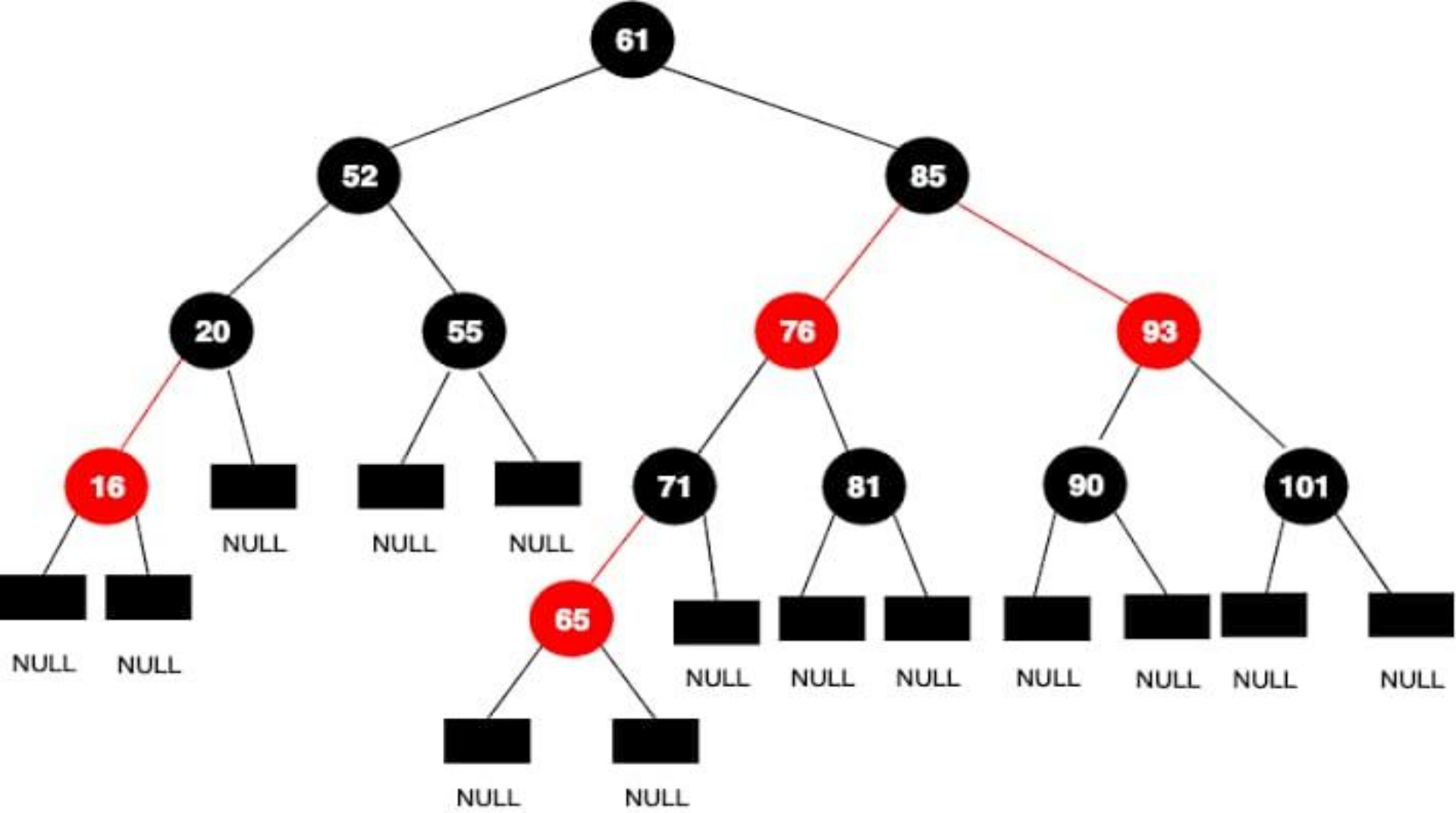
5 **Altura Negra Uniforme** - Todos os caminhos de um nó às suas folhas têm o mesmo número de nós pretos



Black-Height (Altura Preta)

Definição: o número de nós PRETOS em qualquer caminho de um nó até uma folha NIL (sem contar o próprio nó). Denotado: $bh(x)$.





Por Que Usar Árvores Rubro-Negras?

Sistemas Operacionais

Linux usa RBT para gerenciar processos, memória virtual e scheduler CFS

Bancos de Dados

Índices em memória — busca $O(\log n)$ garantida

Linguagens de Programação

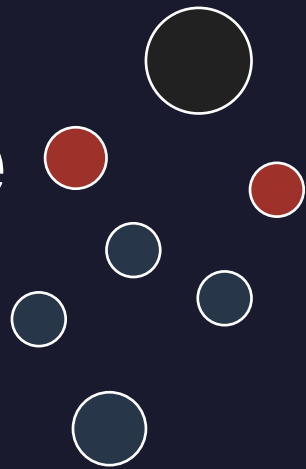
C++ `std::map` e `std::set` / Java `TreeMap` e `TreeSet`

Roteadores de Rede

Tabelas de roteamento IP — inserção/busca rápida

Operações e Complexidade

Chris (Rotações) · Rildo (Inserção, Remoção e Complexidade)



Rotações

- As rotações ocorrem quando uma inserção ou remoção causa desequilíbrio na árvore ou viola alguma das prioridades de coloração das árvores rubro-negras;
- Ao fim de uma rotação, as propriedades de uma árvore binária de busca são mantidas, mantendo a eficiência das operações em tempo $O(\log n)$.

Tipos de Rotações

Existem 2 tipos de rotações simples:

- **RR** (Rotação a esquerda):
- **LL** (Rotação a direita):

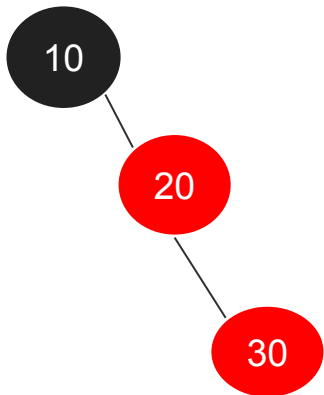
Obs: Em situações mais complexas, elas podem ser combinadas para formar rotações RL ou LR

Rotação RR - (Rotação à Esquerda)

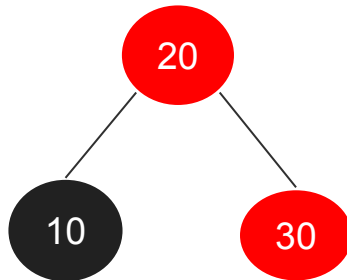
É feita da seguinte forma:

- O filho direito sobe para posição do nó atual;
- O nó atual passa a ser filho esquerdo do nó que subiu;
- Ajuda a reduzir a altura da árvore;
- São feitas mudanças nas cores dos nós.

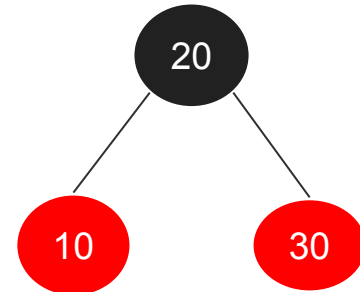
Árvore inicial;



Árvore após rotação



Árvore após recoloração

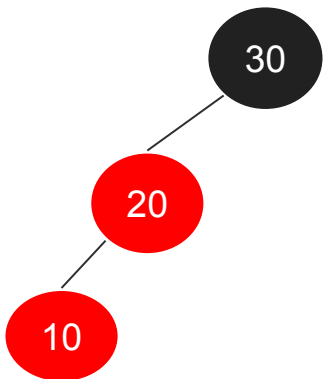


Rotação LL - (Rotação à Direita)

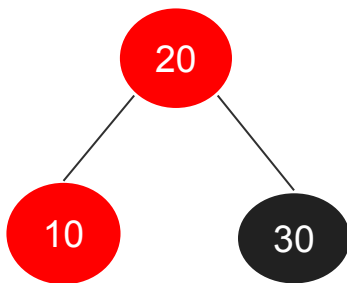
É feita da seguinte forma:

- O filho esquerdo sobe para a posição do nó atual;
- O nó atual passa a ser filho direito do nó que subiu;
- Ajuda a reduzir a altura da árvore.

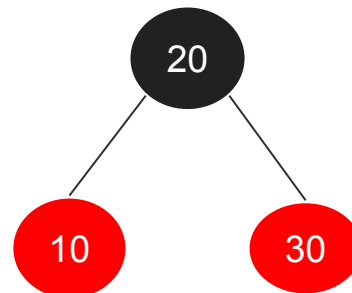
Árvore inicial:



Árvore após rotação:



Árvore após recoloração:



Inserção: Passo a Passo

Passo 1: Inserir como em ABB normal → colorir o novo nó de VERMELHO.

Passo 2: Corrigir violações das propriedades (Prioridade 4: vermelho-vermelho).

Caso 1

Tio do novo nó é VERMELHO

Recolorir: pai e tio → Preto;
avô → Vermelho.
Subir o problema para o avô.

Caso 2

Tio é PRETO e nó é filho DIREITO

Rotacionar o PAI para a esquerda.
Transforma no Caso 3.

Caso 3

Tio é PRETO e nó é filho ESQUERDO

Recolorir pai e avô;
Rotacionar avô para a direita.

Remoção: Visão Geral

A remoção é a operação mais complexa. Ela segue 3 etapas:

1

Remoção BST padrão

Remova o nó como em uma ABB normal.
Se tiver 2 filhos, substitua pelo sucessor in-order.

2

Verificar violação

Se o nó removido (ou substituído) era PRETO, a "black-height" pode ter sido violada → surge o conceito de "nó duplo-preto".

3

Corrigir RB-Delete-Fixup

4 casos (+ simétricos) para restaurar as propriedades usando rotações e recolorações.

Análise de Complexidade

Todas as operações fundamentais têm complexidade $O(\log n)$ — GARANTIDA, não amortizada.

Operação	Melhor caso	Pior caso	Rotações
Busca	$O(\log n)$	$O(\log n)$	$O(1)$
Inserção	$O(\log n)$	$O(\log n)$	$O(\log n)$
Remoção	$O(\log n)$	$O(\log n)$	$O(\log n)$
Mín/Máx	$O(\log n)$	$O(\log n)$	—
Sucessor	$O(\log n)$	$O(\log n)$	—

RBT vs AVL: AVL tem busca ligeiramente mais rápida (mais rígida), mas RBT tem inserção/remoção mais rápida (menos rotações). RBT é preferida na prática.

Inserção — Como Funciona

Regra Inicial: todo novo nó entra sempre como VERMELHO.

O Conflito:

Se o pai do novo nó também for VERMELHO, quebramos a Propriedade 4 da árvore.

Como Corrigir? — O algoritmo olha para o TIO do novo nó:

Caso 1 — Tio Vermelho

Apenas mudamos as cores:
Pai e Tio → PRETO | Avô → VERMELHO
O problema sobe para o avô (recursão).

Casos 2 e 3 — Tio Preto

Mudamos as cores E fazemos rotações
(para esquerda ou direita) para reequilibrar.
Caso 2 vira Caso 3 com uma rotação extra.

Remoção — Como Funciona (Rildo)

A Complexidade: remover um nó VERMELHO é fácil, mas remover um nó PRETO quebra o equilíbrio de caminhos.

O "Duplo Preto"

Quando um nó preto é removido, criamos temporariamente um nó com peso duplo para não perder a contagem de nós pretos nos caminhos.

Esse conceito é resolvido pelo algoritmo RB-Delete-Fixup.

Como Resolver?

- 1 O algoritmo analisa o IRMÃO do nó removido
- 2 Dependendo da cor do irmão e dos sobrinhos, realiza recolorações ou rotações
- 3 4 casos principais (+ simétricos) para restaurar o equilíbrio



Resultado: A árvore se reorganiza sozinha e volta a ficar perfeitamente balanceada.

Complexidade total da remoção: $O(\log n)$ — no máximo 3 rotações necessárias.

Por que a Velocidade é $O(\log n)$? (Rildo)

O que significa $O(\log n)$?

Significa que o tempo de execução cresce de forma extremamente lenta em relação ao número de dados.

1.000 elementos

≤ 10 operações

1.000.000 elementos

≤ 20 operações

1.000.000.000 elementos

≤ 30 operações

Por que isso acontece?

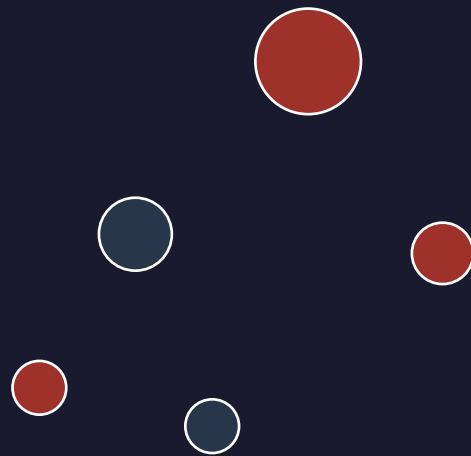
Como a árvore é balanceada pelas regras de cores, cada decisão (esquerda ou direita) elimina metade dos elementos restantes — exatamente como uma busca binária, mas com garantia de balanceamento.

Garantia: inserir, remover e buscar sempre levarão tempo logarítmico — mesmo no pior cenário.

AULA 3

Implementação

Rildo · Adilson — Código e GitHub



Estrutura: Classe Nó

Começamos definindo o nó com seus atributos em Python:

no.py

```
class No:

    def __init__(self, chave):

        self.chave = chave

        self.cor = "VERMELHO"      # novo nó = vermelho

        self.esquerda = None

        self.direita = None

        self.pai = None
```



Decisões de Projeto

- ▶ Os nós NIL teóricos são representado diretamente por None (sem nó sentinela)
- ▶ Todo novo nó nasce com a cor VERMELHA.
- ▶ Ponteiro “pai” ajuda as rotações

Implementação: Rotação à Esquerda

```
def rotacao_esquerda(self, x):  
    y = x.direita          # Filho direito sobe para a posição de x  
    x.direita = y.esquerda  # Subárvore esquerda de y passa a ser direita de x  
    if y.esquerda:  
        y.esquerda.pai = x  
    y.pai = x.pai          # y assume o pai de x  
    if not x.pai:          # Se x era a raiz, y passa a ser a nova raiz  
        self.raiz = y      # x era a raiz  
    elif x == x.pai.esquerda:  
        x.pai.esquerdo = y  
    else:  
        x.pai.direita = y  
    y.esquerda = x          # x fica à esquerda de y  
    x.pai = y
```

Implementação: Rotação à Direita

```
def rotacao_direita(self, y):  
    x = y.esquerda          # Filho esquerdo sobe para a posição de y  
    y.esquerda = x.direita   # Subárvore direita de x passa a ser esquerda de y  
    if x.direita:  
        x.direita.pai = y  
    x.pai = y.pai           # x assume o pai de y  
    if not y.pai:           # Se y era a raiz, x passa a ser a nova raiz  
        self.raiz = x  
    elif y == y.pai.direita:  
        x.pai.direita = x  
    else:  
        x.pai.esquerda = x  
    x.direita = y           # y passa a ser filho direito de x  
    y.pai = x
```

Implementação: Inserção

Inserção seguindo as regras da árvore binária de busca

```
def inserir(self, chave):
```

```
    novo = No(chave)
```

```
    pai = None
```

```
    atual = self.raiz
```

Procura a posição correta para inserir

```
    while atual:
```

```
        pai = atual
```

```
        if novo.chave < atual.chave:
```

```
            atual = atual.esquerda
```

```
        else:
```

```
            atual = atual.direita
```

```
    novo.pai = pai
```

Caso a árvore esteja vazia

```
    if pai is None:
```

```
        self.raiz = novo
```

Inserção à esquerda

```
    elif novo.chave < pai.chave:
```

```
        pai.esquerda = novo
```

Inserção à direita

```
    else:
```

```
        pai.direita = novo
```

Corrige possíveis violações da árvore
rubro-negra

```
    self.corrigir_insercao(novo)
```

Implementação: Rebalanceamento após inserção

```
def corrigir_insercao(self, z):  
  
    # Executa enquanto houver dois nós vermelhos  
    consecutivos  
    while z != self.raiz and z.pai.cor == "VERMELHO":  
  
        # Caso em que o pai está à esquerda do avô  
        if z.pai == z.pai.pai.esquerda:  
  
            # Identifica o tio  
            tio = z.pai.pai.direita  
  
            # Caso 1: tio vermelho  
            if tio and tio.cor == "VERMELHO":
```

```
                # Recoloração  
                z.pai.cor = "PRETO"  
                tio.cor = "PRETO"  
                z.pai.pai.cor = "VERMELHO"  
                # Continua verificando a partir do avô  
                z = z.pai.pai  
            else:  
  
                # Caso 2: nó é filho direito  
                if z == z.pai.direita:  
                    z = z.pai  
                    self.rotacao_esquerda(z)  
  
                # Caso 3: rotação e recoloração  
                z.pai.cor = "PRETO"  
                z.pai.pai.cor = "VERMELHO"  
  
                self.rotacao_direita(z.pai.pai)
```


Implementação: Caso simétrico (pai está à direita do avô)

else:

```
tio = z.pai.pai.esquerda
```

```
# Caso 1: tio vermelho
```

```
if tio and tio.cor == "VERMELHO":
```

```
    z.pai.cor = "PRETO"
```

```
    tio.cor = "PRETO"
```

```
    z.pai.pai.cor = "VERMELHO"
```

```
    z = z.pai.pai
```

else:

```
# Caso 2: nó é filho esquerdo
```

```
if z == z.pai.esquerda:
```

```
    z = z.pai
```

```
    self.rotacao_direita(z)
```

```
# Caso 3: rotação e recoloração
```

```
z.pai.cor = "PRETO"
```

```
z.pai.pai.cor = "VERMELHO"
```

```
self.rotacao_esquerda(z.pai.pai)
```

```
# Garante que a raiz seja sempre preta
```

```
self.raiz.cor = "PRETO"
```

Implementação: Buscar um valor na árvore

```
# Busca um valor na árvore
def buscar(self, chave):

    atual = self.raiz

    while atual:

        if chave == atual.chave:
            return atual

        elif chave < atual.chave:
            atual = atual.esquerda

        else:
            atual = atual.direita

    return None
```

Implementação: Remover um valor da árvore

Remove um valor da árvore

```
def remover(self, chave):
```

```
    no = self.buscar(chave)
```

Valor não encontrado

```
    if no is None:
```

```
        return False
```

Caso 1: nó sem filhos

```
    if no.esquerda is None and no.direita is None:
```

```
        if no == self.raiz:
```

```
            self.raiz = None
```

```
        elif no == no.pai.esquerda:
```

```
            no.pai.esquerda = None
```

```
        else:
```

```
            no.pai.direita = None
```

Implementação: Remover um valor da árvore

Caso 2: nó possui apenas filho direito

elif no.esquerda is None:

self.substituir(no, no.direita)

Caso 3: nó possui apenas filho esquerdo

elif no.direita is None:

self.substituir(no, no.esquerda)

Caso 4: nó possui dois filhos

else:

sucessor = self.minimo(no.direita)

no.chave = sucessor.chave

if sucessor == sucessor.pai.esquerda:

sucessor.pai.esquerda = sucessor.direita

else:

sucessor.pai.direita = sucessor.direita

return True

Implementação: Substituir um nó por outro

```
def substituir(self, antigo, novo):  
  
    if antigo.pai is None:  
        self.raiz = novo  
  
    elif antigo == antigo.pai.esquerda:  
        antigo.pai.esquerda = novo  
  
    else:  
        antigo.pai.direita = novo  
  
    if novo:  
        novo.pai = antigo.pai
```

Implementação: Retornar o menor nó de uma subárvore

```
def minimo(self, no):  
    while no.esquerda:  
        no = no.esquerda  
  
    return no
```

Implementação: Percurso em ordem

```
def em_ordem(self, no):  
    if no:  
        self.em_ordem(no.esquerda)  
        print(f"{no.chave} ({no.cor})")  
        self.em_ordem(no.direita)
```

Implementação: Mostrar árvore

```
def mostrar_arvore(self, no, espaco="", ultimo=True):
```

```
    if no is not None:
```

```
        print(espaco, end="")
```

```
        if ultimo:
```

```
            print("  ", end="")
```

```
            novo_espaco = espaco + "  "
```

```
        else:
```

```
            print(" | ", end="")
```

```
            novo_espaco = espaco + "| "
```

```
        print(f"{no.chave} ({no.cor[0]})")
```

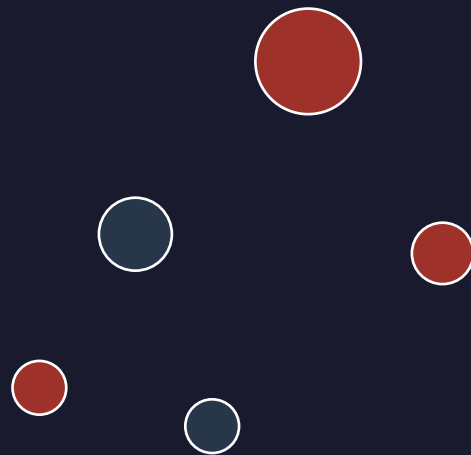
```
        self.mostrar_arvore(no.esquerda,  
                             novo_espaco, False)
```

```
        self.mostrar_arvore(no.direita, novo_espaco,  
                             True)
```


AULA 4

Prática: Hands-on

Exercícios, quiz interativo e encerramento — Perla



Recursos da Aula Prática

Repositório de Código

[github.com/JAdilsonSA/
-rvore-RN](https://github.com/JAdilsonSA/-rvore-RN)

O repositório contém:

- ▶ Implementação completa em Python
- ▶ Comentários explicativos em cada função
- ▶ Código de teste com inserções e remoções
- ▶ README com instruções de uso

Quiz Interativo — Wayground

[wayground.com/admin/quiz/
6a2ebd7486925a22f2ecd4a0](https://wayground.com/admin/quiz/6a2ebd7486925a22f2ecd4a0)

Como participar:

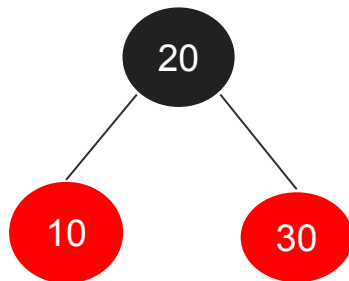
- ▶ Acesse o link acima pelo celular
- ▶ Responda as questões em tempo real
- ▶ Veja o ranking ao vivo
- ▶ Gabarito comentado no final



Abra o GitHub para acompanhar o código ao vivo e o Wayground para o quiz final!

Exercício 1 — INSERÇÃO

Árvore inicial:



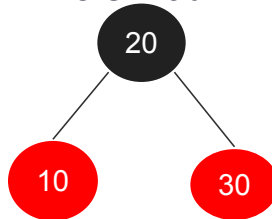
Insira os valores na ordem: 5 e 15

O que fazer:

- 1 Inserir seguindo as regras da árvore rubro-negra
- 2 Corrigir violações (se houver)
- 3 Mostrar a árvore final balanceada

Exercício 1 — INSERÇÃO(Correção)

Árvore inicial:



Insira os valores 5 e 15

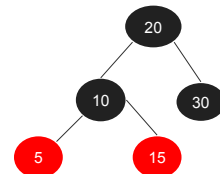
Correção (Recoloração)

- 10 → vira preto
- 30 → vira preto
- 20 → vira vermelho
- Como 20 é raiz, volta para preto

Inserindo o 15:

- 15 < 20 → vai para esquerda
- 15 > 10 → fica como filho direito

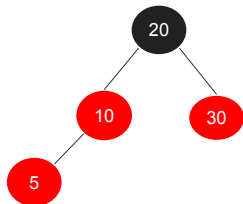
Resultado final da árvore com o 15:



Inserindo o 5:

- 5 < 20 → vai para esquerda
- 5 < 10 → fica como filho esquerdo do 10

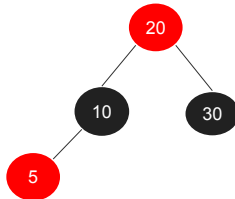
Resultado parcial da árvore com o 5:



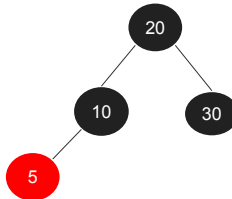
Problema:

- Pai (10) é vermelho
- Tio (30) é vermelho
- caso de recoloração

Após recoloração:



Raiz volta a ser preta:



Uma Árvore Rubro-Negra segue 5 regras:

1. Todo nó é vermelho ou preto.
2. A raiz é preta.
3. Todas as folhas NIL são pretas.
4. Nó vermelho não pode ter filho vermelho.
5. Todos os caminhos da raiz até NIL possuem a mesma quantidade de nós pretos.

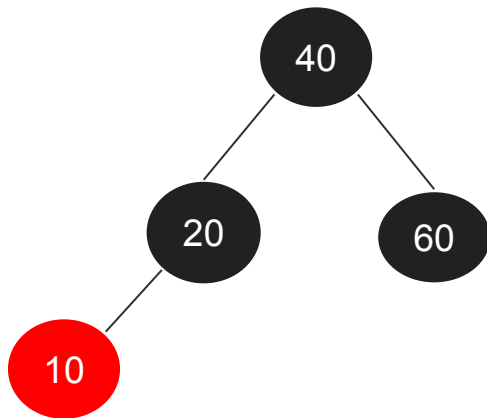
Exercício 2 — Inserção e Rotação

Insira o valor na ordem: 5

O que fazer:

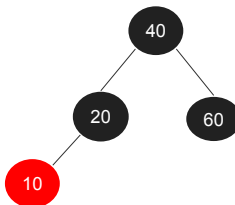
- Inserir seguindo as regras da árvore Rubro-Negra
- Identificar se há violações
- Aplicar a rotação necessária
- Corrigir as cores
- Mostrar a árvore final

Árvore inicial:



Exercício 2 — INSERÇÃO E ROTAÇÃO(Correção)

Árvore inicial:



Inserindo o 5:

- $5 < 40 \rightarrow$ vai para esquerda
- $5 < 20 \rightarrow$ vai para esquerda
- $5 < 10 \rightarrow$ fica como filho esquerdo do 10

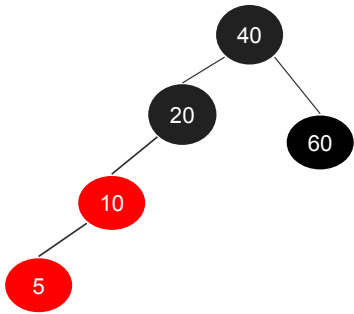
Problema após inserir o 5:

- 10 (R) com filho 5 (R)
→ violação vermelho-vermelho
- Caso LL: rotação à direita

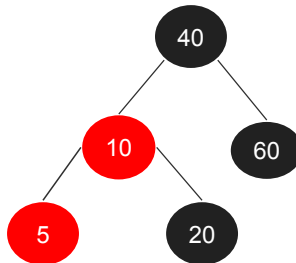
Árvore final com correção:

- Correção Rotação à direita em 20.
(10 sobe e 20 desce para a direita)
- Relocação: 10 (R) vira preto, 20 (B) vira vermelho

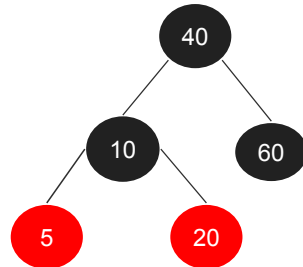
Resultado parcial da árvore com o 5:



Resultado com rotação



Resultado final com a rotação



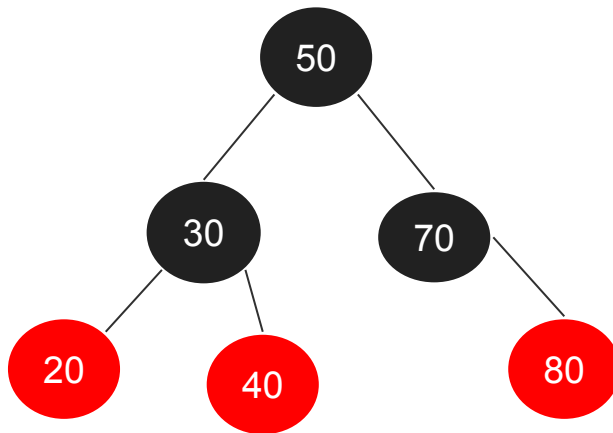
Exercício 3 — REMOÇÃO

Remova nessa ordem: 30 → 70

O que fazer:

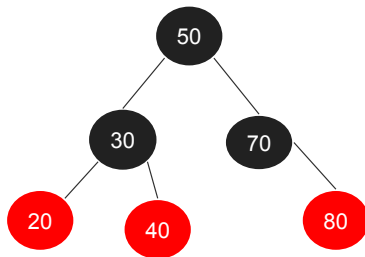
- Remover respeitando as regras da árvore rubro-negra.
- Tratar casos de : nó duplo e reestruturação da árvore
- Manter propriedades da árvores

Árvore inicial:



Exercício 3 — REMOÇÃO(Correção)

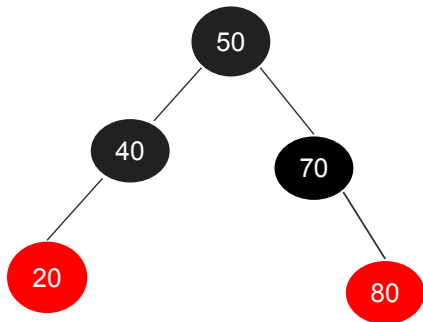
Árvore inicial:



Remoção do 30

- 30 tem dois filhos
- substitui pelo menor sucessor a direita (40)
- 40 sobe no lugar do 30

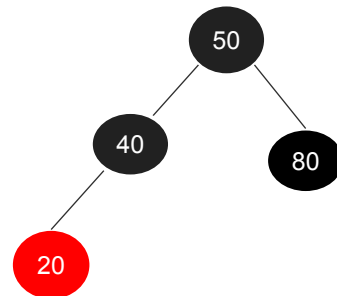
Resultado parcial da árvore com remoção do 30 :



Remoção do 70

- 70 é preto e possui um filho vermelho, remoção simples;
- O filho substitui o pai
- O filho assume a cor do nó removido

Resultado da árvore final com remoção do 70:



Obrigado!

Dúvidas e discussões?

✓ Propriedades garantem $O(\log n)$

✓ 3 casos de correção na inserção

✓ Rotações: $O(1)$ — base do equilíbrio

✓ Usado em Linux, C++ STL, Java JDK

Equipe 3 — Estruturas de Dados II